Coding with Kanak

Data Warehouse Implementation & Data Cube Computation

Unit 4

Data Warehousing: Unit-4

Data warehouse Implementation

Data warehouse implementation refers to the process of designing, building and deploying a centralized system that stores, manages and i ntegrates data from various sources to support business analysis and decision-making.

The purpose of implementing a data warehouse i s to provide organizations, a unified repository of historical data, enabling efficient querying, reporting and data analysis.

Proper implementation is crucial for businesses because it ensures the system meets the organization's needs for scalability, data accuracy and performance.

A well-implemented data warehouse provides key benefits such as improved decision -making, streamlined data access, enhanced reporting capabilities and better data consistency, all of which contribute to data-driven business success.

4.1 Steps in Data Warehouse Implementation

1. Planning and Requirements Gathering: Understand business needs, set objectives and determine hardware/software requirements.

2. Data Modeling and Design: Choose the appropriate schema (e.g., Star, Snowflake) to optimize data organization and query performance.

3. ETL Process: Extract, Transform and Load: Extract, clean, transform and load data, ensuring consistency and accuracy, with customized ETL tools.

4. Database Design and Architecture: Design the physical architecture, including storage, indexing and optimization for efficient performance.

5. Data Warehouse Dev elopment: Create tables, views and other objects, ensuring scalability for future data growth.

6. Testing and Validation: Verify data accuracy and performance, ensuring the system meets requirements and resolving any discrepancies.

7. Deployment and Maintenance: Deploy the system, address issues, apply updates and maintain continuous data integration.

1.2 Data Warehouse Physical Design

Physical database design translates a logical design into a physical one by choosing specific storage structures, data types, and indexing strategies to meet performance and maintenance requirements.

Key steps involve creating tables, defining storage and partitioning, and optimizing queries and data access.

4.2.1 Steps

Translate the logical model:

Convert the logical design into the specific syntax of the target database system, using SQL statements.

Create tables and define structures:

Define tables, the basic unit for data storage, and select appropriate data types for columns.

Plan for storage and partitioning:

Decide how data will be physically stored and consider partitioning large tables to improve management and performance.

Design indexes:

Choose or create indexes to speed up data retrieval by making common queries faster.

Enforce data integrity:

Apply constraints to ensure data validity and consistency, which can be a different process for different database types (e.g., OLTP vs. data warehouses).

Refine schema:

Make final adjustments, which may include denormaliz ation in specific cases to improve read performance, even if it introduces some redundancy.

4.2.2 Considerations

Considerations include balancing storage space and data integrity, planning for scalability, and enforcing data integrity through constraints.

Performance optimization:

This is a primary driver for physical design, focusing on minimizing disk I/O and optimizing join operations.

Scalability:

The design should be able to handle future growth in data volume and user load.

Physical storage:

Choose storage formats and arrangements that balance storage space requirements with performance needs.

Data integrity:

The design must include mechanisms to enforce business rules and prevent invalid data from being entered.

Security:

Plan for security at the physical level.

Maintenance:

Consider how the design will impact future maintenance tasks, such as backups and upgrades.

4.2.3 Physical storage

Tables:

The fundamental building blocks for storing data.

Partitioning:

Dividing large tables into smaller, more manageable pieces based on specific criteria to improve performance and manageability.

Storage formats:

Select the most appropriate storage format for your data to optimize for space and speed.

Clustering:

Grouping related data together on disk to reduce the need for disk I/O when accessing it.

4.2.4 Indexing

Indexes:

Structures that improve the speed of data retrieval operations.

Index selection:

Choosing which columns to index based on how they are used in queries.

Index optimization:

Tuning the indexes to ensure they are effective and not creating unnecessary overhead.

Types of indexes:

Selecting the right type of index (e.g., B-tree, hash) for different use cases.

4.2.5 Performance optimization

Query optimization:

Analyzing and rewriting queries to make them more efficient.

Data access optimization:

Minimizing the number of disk I/O operations needed to access data.

Denormalization:

In some cases, intentional ly introducing some redundancy into the schema can improve read performance by reducing the need for complex joins.

Caching:

Using caching at the application level to reduce the number of requests to the database.

Monitoring:

Continuously monitoring performance to identify bottlenecks and areas for improvement.

4.3 Data Warehouse Deployment Activities

Data warehouse deployment involves preparing the production environment, installing and configuring the system, testing, and then officially launching it for end -users. Key activities include setting up infrastructure, deploying ETL/ELT tools, configuring the data warehouse and OLAP cubes, and implementing security measures.

The process is followed by a critical phase of ongoing maintenance, monitoring, data refreshing, and performance tuning to ensure continued success.

4.3.1 Pre-deployment and setup

Environment preparation:

Set up the necessary production infrastructure to support the data warehouse.

System configuration:

Install and configure the data warehouse, ETL/ELT tools, and other necessary analytics and reporting tools.

Security implementation:

Implement security controls, access management, and backup/recovery procedures.

Sizing and performance tuning:

Determine the optimal sizing for the system and conduct initial performance tuning.

OLAP cube design:

Build and configure the OLAP cubes for multi-dimensional analysis.

4.3.2 Deployment and launch

Deploy to production:

Move the validated and tested data warehouse to the production environment, making it accessible to users.

Data refresh scheduling:

Schedule and implement regular data updates and incremental load processes to maintain data freshness.

User training:

Train end-users on how to access and use the new data warehouse and its reporting tools.

Go-live:

Officially launch the system for general use.

4.3.3 Post-deployment and maintenance

Monitoring and optimization:

Continuously monitor performance, set up alerts for issues, and optimize query execution and system efficiency.

Change management:

Implement a change management process for all future updates and enhancements.

Ongoing support:

Provide ongoing support to users, including resolving issues and answering questions.

Continuous improvement:

Use user feedback to make ongoing improvements and add new features or data sources as needed.

Backup and recovery:

Conduct regular backup and disaster recovery drills to ensure business continuity.

4.4 Data Cube Computation

Data cube computation is the process of pre -calculating aggregations over multidimensional data to enable fast querie s in online analytical processing (OLAP). It involves creating a lattice of "cuboids," where each cuboid represents a set of aggregated data, such as sales by city, item, and year.

Key algorithms for this include MultiWay array aggregation, which processes chunks of data in memory, and BUC (Bottom -Up-Cubing), which is optimized for iceberg cubes (aggregations that meet a certain threshold).

4.4.1 How it works

Multidimensional data model:

Data is structured in multiple dimensions (e.g., time, product, location) and then aggregated. The result is a "data cube" composed of many "cuboids".

Cuboids and aggregations:

Each cuboid represents an aggregation of data for a specific subset of dimensions. The simplest cuboid (the base cuboid) contains all dimensions, while the most generalized one (the apex cuboid) contains the total sum across all dimensions.

Pre-computation:

The computation step involves performing the aggregations for all or some of these cuboids in advance. This is done to speed up query response times significantly.

Materialization strategies:

Full cube: Pre-computes every possible aggregation. This leads to very fast queries but requires a large amount of storage.

Iceberg cube: Pre-computes only those aggregations that meet a specific threshold, saving storage space.

Shell cube: Pre-computes only a shell fragment of the cube, which can be useful for high-dimensional data.

4.4.2 Case study on Data Computation

We have a database that contains transaction information relating company sales of a part to a customer at a store location. The data cube formed from this database is a 3 -dimensional representation, with each cell (p,c,s) of the cube representing a combi nation of values from part, customer and store-location.

A sample data cube for this combination is shown in Figure 1. The contents of each cell are the count of the number of times that specific combination of values occurs together in the database. Cells that appear blank in fact have a value of zero. The cube can then be used to retrieve information within the database about, for example, which store should be given a certain part to sell in order to make the greatest sales.

Figure 1(a): Front View of Sample Data Cube

Figure 1(b): Entire View of Sample Data Cube

Computed versus Stored Data Cubes

The goal is to retrieve the decision support information from the data cube in the most efficient way possible. Three possible solutions are:

1. Pre-compute all cells in the cube
2. Pre-compute no cells
3. Pre-compute some of the cells

If the whole cube is pre -computed, then queries run on the cube will be very fast. The disadvantage is that the pre -computed cube requires a lot of memory. The size of a cube for n attributes A1,...,An with cardinalities |A 1|,...,|An| is π|A i|. This size increases exponentially with the number of attributes and linearly with the cardinalities of those attributes.

To minimize memory requirements, we can pre -compute none of the cells in the cube. The disadvantage here is that queries on the cube will run more slowly because the cube will need to be rebuilt for each query.

As a compromise between these two, we can pre-compute only those cells in the cube which will most likely be used for decision support queries. The trade -off between memory space and computing time is called the space-time trade -off, and it often exists in data mining and computer science in general.

Representation

m-Dimensional Array:

A data cube built from m attributes can be stored as an m-dimensional array. Each element of the array contains the measure value, such as count. The array itself can be represented as a 1 -dimensional array.

For example, a 2 -dimensional array of size x x y can be stored as a 1 -dimensional array of size x*y, wh ere element (i,j) in the 2 -D array is stored in location (y*i+j) in the 1 -D array. The disadvantage of storing the cube directly as an array is that most data cubes are sparse, so the array will contain many empty elements (zero values).

List of Ordered Sets:

To save storage space we can store the cube as a sparse array or a list of ordered sets. If we store all cells in the data cube from Figure 1, then the resulting datacube will contain (cardPart

• cardStoreLocation*cardCustomer) combinations, which is 5 * 4 * 4 = 80 combinations. If we eliminate cells in the cube that contain zero, such as {P1, Vancouver, Allison}, only 27 combinations remain, as seen in Table 1.

Table 1 shows an ordered set representation of the data cube. Each attribute value combinat ion is paired with its corresponding count. This representation can be easily stored in a database table to facilitate queries on the data cube.

Combination Count

{P1, Calgary, Vance} 2

{P2, Calgary, Vance} 4

{P3, Calgary, Vance} 1

{P1, Toronto, Vance} 5

{P3, Toronto, Vance} 8

{P5, Toronto, Vance} 2

{P5, Montreal, Vance} 5

{P1, Vancouver, Bob} 3

{P3, Vancouver, Bob} 5

{P5, Vancouver, Bob} 1

{P1, Montreal, Bob} 3

{P3, Montreal, Bob} 8

{P4, Montreal, Bob} 7

{P5, Montreal, Bob} 3

{P2, Vancouver, Richard} 11

{P3, Vancouver, Richard} 9

{P4, Vancouver, Richard} 2

{P5, Vancouver, Richard} 9

{P1, Calgary, Richard} 2

{P2, Calgary, Richard} 1

{P3, Calgary, Richard} 4

{P2, Calgary, Allison} 2

{P3, Calgary, Allison} 1

{P1, Toronto, Allison} 2

{P2, Toronto, Allison} 3

{P3, Toronto, Allison} 6

{P4, Toronto, Allison} 2

Table 1: Ordered Set Representation of a Data Cube

Representation of Totals

Another aspect of data cube representation which can be considered is the representation of totals. A simple data cube does not contain totals. The storage of totals increases the size of the data cube but can also decrease the time to make total-based queries.

A simple way to represent totals is to add an additional layer on n sides of the n-dimensional datacube. This can be easily visualized with the 3 -dimensional data cube introduced in Figure 1. Figure 2 shows the original cube with an additional laye r on each of three sides to store total values.

The totals represent the sum of all values in one horizontal row, vertical row (column) or depth

row of the data cube.

Figure 2: Cube with Total

4.4.3 Common computation algorithms

MultiWay array aggregation:

Partitions large, multidimensional arrays into smaller, manageable "chunks" that can be processed in memory. It performs simultaneous aggregation across multiple dimensions, which is efficient when the product of the cardinalities of the dimensions is moderate.

BUC (Bottom-Up-Cubing):

Explores the use of sorting and ordering to efficiently compute iceberg cubes, typically using a top-down approach.

Star-Cubing:

Integrates both top -down and bottom -up computation methods by using a star-tree structure.

Apriori pruning:

A method borrowed from association rule mining that can be applied to efficiently compute iceberg cubes by pruning the computation of descendants that do not meet a minimum support threshold.

4.4 Indexing OLAP Data

One of the key factors that affect OLAP performance is indexing, which is the process of creating and maintaining structures that help the database engine to locate and retrieve data faster.

4.4.1 OLAP Indexing:

OLAP indexing is a specific type of indexing that is designed for OLAP databases, which store data in a multidimensional format, such as cubes, dimensions, and hierarchies. OLAP indexing creates and maintains structures that help the database engine naviga te and access the data in these multidimensional objects, as well as perform calculations and aggregations on them. OLAP indexing is important because it can significantly improve the performance of OLAP queries and reports, which are often complex and resource-intensive.

OLAP queries and reports typically involve slicing and dicing the data along multiple dimensions, applying filters and conditions, and calculating measures and indicators. Without proper OLAP indexing, these operations can take a long ti me and consume a lot of memory and CPU resources.

4.4.2 OLAP Indexing Types:

OLAP indexing can be done at different levels of granularity, depending on the needs and characteristics of the data and the queries. OLAP indexing can also be done automaticall y by the database engine, or manually by the database developer or administrator.

Bitmap indexes are well-suited to low-cardinality columns or dimensions such as gender, status, or category. These indexes are efficient for OLAP queries and can support log ical operations, compression, and encoding techniques.

B-tree indexes are better for high -cardinality columns or dimensions like dates, IDs, or names. They are able to support range queries, prefix searches, and sorting operations.

Join indexes store the results of join operations between two or more tables or cubes, which can eliminate the need to perform the join at run time. These indexes are useful for complex or expensive join conditions and can support fast filtering and aggregation operations on the joined data.

4.5 Efficient Processing of OLAP Query

Efficient processing of OLAP queries in a data warehouse involves several key techniques and considerations:

4.5.1 Materialized Views and Cuboids:

Pre-compute and store aggregated data in materialized views or cuboids (pre - calculated summary tables) for frequently accessed aggregations. This reduces the need to perform complex computations at query time.

Careful selection of which cuboids to materialize is crucial, considering storage costs and query frequency.

4.5.2 Indexing Strategies:

Bitmap Indexes:

Efficient for low -cardinality dimensions, offering fast retrieval and space efficiency compared to B-tree indexes.

Bitmap Join Indexes:

Integrate join indexes with bitmap indexes for efficient star join operations, common in data warehouses.

Measure Attribute (MA) Index:

A specialized index structure that can support index -only processing for star joins and grouping operators, significantly improving query performance.

4.5.3 Query Optimization and Execution:

Query Rewriting:

Transform OLAP operations (slice, dice, roll-up, drill-down) into optimized SQL queries.

Cuboid Selection:

When multiple materialized cuboids could answer a query, select the most appropriate one based on dominance relationships and estimated execution cost.

Cost-Based Optimization:

Estimate the cost of executing operations on different cuboids and choose the one with the least cost.

4.5.4 Data Partitioning:

Partitioning fact tables, especially large ones, can improve query performance by reducing the amount of data scanned and facilitating parallel query execution.

Horizontal fragmentation based on dimensions like 'Location' can enhance local query processing speed in distributed data warehouses.

4.5.5 Distributed OLAP Query Processing:

In distributed data warehouses, techniques like local OLAP query prioritization, based on query logs and cuboid access patterns, can reduce the load on the central warehouse.

Optimized strategies for executing OLAP queries across cuboids stored in a distributed environment are crucial for performance.

4.5.6 Hardware and Software Optimization:

Leverage powerful hardware, including sufficient RAM and fast storage, to support large -scale data processing.

Utilize database management systems and OLAP tools designed for high -performance analytical processing.

4.6 OLAP Server Architectures

An OLAP (Online Analytical Processing) server architecture organizes and processes data in a multidimensional format using structures called "cubes" or "hypercubes". The architecture typically involves a user interface, an OLAP server with its own storage and processing engine, and a data source (often a data warehouse).

4.6.1 Core components

Client Application:

The user -facing part, which could be a web application, reporting tool, or API, that sends analytical queries to the server.

OLAP Server:

The central processing engine that stores data in multidimensional cubes and performs the analysis. It accepts requests from the client, processes them, and returns results.

Data Warehouse:

The underlying source of data, which is often a relational database organized in a star or snowflake schema. The OLAP server retrieves data from here to build its cubes.

4.6.2 Key characteristics

Key features include organizing data by dimensions (like time or geography) and measures (like sales), pre -calculating aggregations for fast queries, and supporting operations like drill-down, roll-up, slice, and pivot.

Multidimensional Data Model:

Data is organized into dimensions (e.g., time, location, product) and measures

(e.g., sales, quantity), which are stored in multidimensional "cubes".

Pre-calculated Aggregations:

To ensure fast query performance, OLAP servers pre -calculate and store aggregated data for various combinations of dimensions.

Hierarchies:

Dimensions can have hierarchies (e.g., year > quarter > month) allowing users to analyze data at different levels of detail.

Analytical Operations:

Users can perform operations like:

Drill-down: Moving from a summary to more detailed data.

Roll-up: Moving from detailed data to a higher-level summary.

Slice: Selecting a single value for one dimension to create a new sub -cube.

Pivot (or Rotate): Changing the orientation of a cube to view it from a different perspective.

4.6.3 Types of OLAP architectures

MOLAP (Multidimensional OLAP):

Stores data directly in a specialized multidimensional database, providing the fastest query performance.

ROLAP (Relational OLAP):

Stores data in a relational database and uses a relational engine to handle multidimensional queries. It is less performant than MOLAP but is more scalable.

HOLAP (Hybrid OLAP):

A hybrid approach that combines MOLAP and ROLAP. It stores pre-aggregated data in a multidimensional format and the detailed data in a relational database , aiming to balance performance and scalability.

References:

[1]: https://www.scaler.com/topics/data-mining-tutorial/what-is-olap/

[2]: https://www.shikhadeep.com.np/2021/12/olap-architecture.html

[3]: https://bytehouse.cloud/blog/rolap-molap-holap

[4]: https://www.janbasktraining.com/tutorials/implementation-of-data-warehouse/

[5]: https://www.guvi.in/blog/data-warehousing-concepts-and-implementation/

[6]: https://www.geeksforgeeks.org/dbms/multi-tier-architecture-of-data-warehouse/

[7]: https://oracle-datawarehousing.blogspot.com/2011/02/physical-design-in-data-warehouses.html

[8]: https://www.projectpro.io/article/how-to-design-a-data-warehouse/825

[9]: https://momentum-tech.ca/en/launching-a-data-warehouse-project-where-to-start/

[10]: https://fastercapital.com/topics/data-warehouse-maintenance-and-performance-optimization.html/1

[11]: https://www.geeksforgeeks.org/data-analysis/data-warehouse-development-life-cycle-model/

[12]: https://www2.cs.uregina.ca/~dbd/cs831/notes/dcubes/dcubes.html

[13]:https://www.studocu.com/in/messages/question/3128990/explain-olap-data-indexing-for-bitmap-index-and-

join-index

[14]:https://www.sciencedirect.com/topics/computer-science/physical-database-design#:text-=Physical%20-

Database%20Design%20refers%20to%20the%20process,data%20efficiently.%20How%20useful%20is%20this%2

0definition?